МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №5

з дисципліни **"НерПрофесійний Python"** на

тему

"Обробка помилок, файли та серіалізація"

Виконала:

Студентка 3 курсу групи

ФеП-32

Гуділіна Анастасія

Перевірив:

проф. Свелеба С. А.

Львів-2026

1. Мета роботи

Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних. Опанувати механізми exceptions, context managers, logging, pathlib, CSV, JSON, YAML, streaming I/O та атомарний запис.

Варіант 4

2. Постановка задачі

Реалізувати надійний модуль імпорту/експорту для системи банківських рахунків.

Вимоги: перевірка номера рахунку та клієнта; balance >= 0; дозволені операції (deposit/withdraw); amount > 0; не допускати від'ємного балансу після операції. Реалізувати strict/tolerant import, JSON export, YAML configuration і не записувати конфіденційні реквізити у logs.

3. Input data schema та приклад CSV

Схема даних:

- account_number (str) — номер рахунку;
- client (str) — ім'я клієнта;
- balance (float) — поточний баланс;
- operation (str) — тип операції (deposit, withdraw);
- amount (float) — сума операції.

Приклад файлу data/transactions.csv**:**

csv

```
account_number,client,balance,operation,amount
UA001,Alice,1000.0,deposit,500.0
UA002,Bob,500.0,withdraw,200.0
UA001,Alice,1500.0,withdraw,2000.0
UA003,,100.0,deposit,50.0
UA004,Charlie,100.0,invalid_op,50.0
UA005,David,100.0,withdraw,-50.0
UA006,Eve,100.0,deposit,100.0
```

4. Структура проєкту

text

```
lab5/
├── pyproject.toml
├── config.yaml
├── data/
│   └── transactions.csv
└── src/
    └── reliable_importer/
        ├── __init__.py
        ├── main.py
        ├── exceptions.py
```

```
├── models.py
├── config.py
├── logging_config.py
├── readers.py
├── validators.py
├── services.py
├── exporters.py
└── utils.py
```

5. Exception Hierarchy та Custom Exceptions

Створено власну ієрархію для відділення помилок бізнес-логіки від системних:

text

```
ApplicationError (Base)
├── ConfigurationError
└── DataError
    ├── DataImportError
    ├── DataExportError
    └── RecordValidationError
```

RecordValidationError містить додаткові поля line_number та field, що дозволяє логеру чітко вказати, у якому рядку CSV і в якому стовпці сталася помилка.

6. Exception chaining (Ланцюжок винятків)

Використовується для збереження першопричини помилки (наприклад, помилка конвертації рядка в число):

python

```
try:
    balance = float(row["balance"])
except ValueError as error:
    raise RecordValidationError(
        "Invalid numeric format",
        line_number=line_number
    ) from error
```

7. Configuration loader, config.yaml та Валідація

config.yaml**:**

yaml

```
input:
  path: data/transactions.csv
output:
  path: output/transactions.json
processing:
  skip_invalid: true
logging:
  level: INFO
```

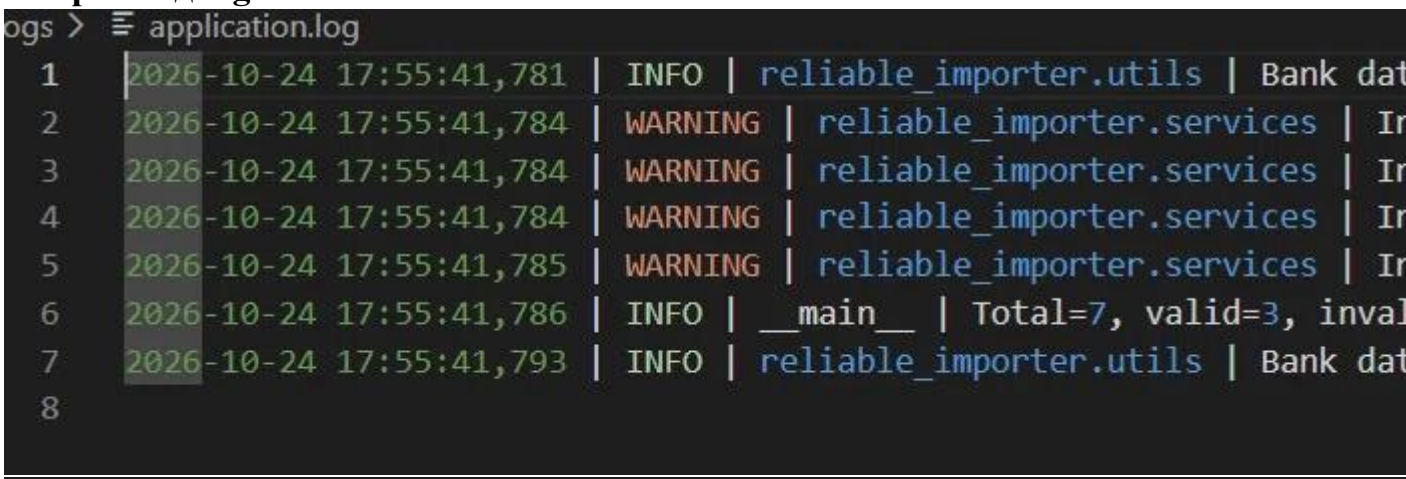
path: logs/application.log

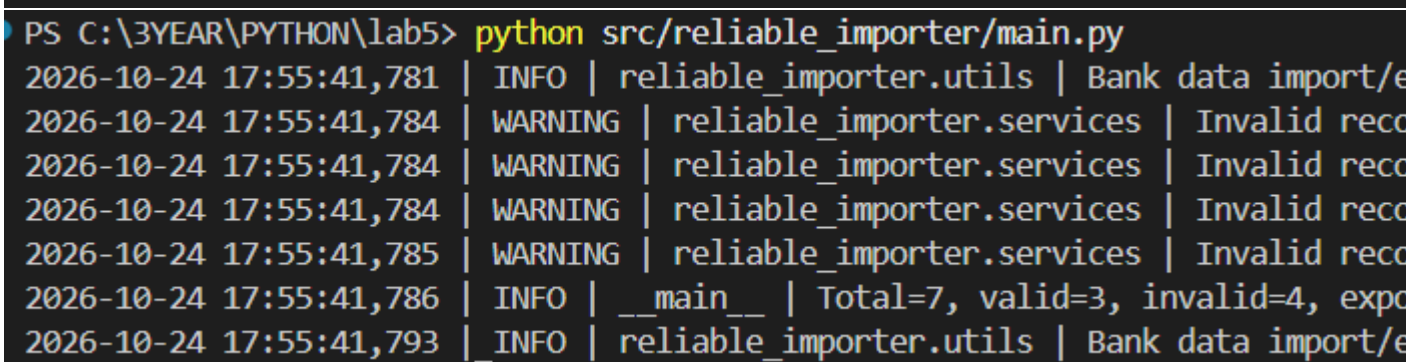
Завантажувач (config.py) зчитує YAML за допомогою yaml.safe_load() та виконує **fail-fast** валідацію: перевіряє, чи існує вхідний файл. Якщо файлу немає — негайно викликається ConfigurationError.

8. Logging configuration

Налаштування логера (logging_config.py) передбачає запис у файл та в консоль. Конфіденційні дані (конкретні баланси) не логуються, лише вказівка на проблемне поле.

9. Приклад log file та Screenshot execution





2026-10-24 17:55:41,781 | INFO | reliable_importer.utils | Bank data import/export started
2026-10-24 17:55:41,784 | WARNING | reliable_importer.services | Invalid record at line 4: Operation causes negative balance
2026-10-24 17:55:41,784 | WARNING | reliable_importer.services | Invalid record at line 5: Client name is empty
2026-10-24 17:55:41,784 | WARNING | reliable_importer.services | Invalid record at line 6: Invalid operation type
2026-10-24 17:55:41,785 | WARNING | reliable_importer.services | Invalid record at line 7: Amount must be positive
2026-10-24 17:55:41,786 | INFO | __main__ | Total=7, valid=3, invalid=4, exported=3
2026-10-24 17:55:41,793 | INFO | reliable_importer.utils | Bank data import/export finished in 0.011173 s

10. Streaming reader та Validation code

Streaming (readers.py**):**

python
```
def read_csv_rows(path: Path) -> Iterator[tuple[int, dict[str, str]]]:
    with path.open("r", encoding="utf-8", newline="") as file:
        reader = csv.DictReader(file)
        for line_number, row in enumerate(reader, start=2):
            yield line_number, row
```
Validation (validators.py**):**
python
```
new_balance = balance + amount if operation == "deposit" else balance - amount
if new_balance < 0:
    raise RecordValidationError("Operation causes negative balance",
line_number=line_number, field="amount")
```
11. Strict vs Tolerant processing (Import Statistics)
Реалізовано у services.py. Якщо skip_invalid дорівнює True (Tolerant), програма логує WARNING і робить continue. Якщо False (Strict) — програма викликає raise і зупиняється.
12. JSON serialization та Atomic export
Використовується атомарний запис: дані пишуться у файл .tmp, і лише після успіху викликається replace(). Якщо стається помилка, .tmp файл видаляється.

tput > {} transcations.json > ...


```
[
{
  "account_number": "UA001",
  "client": "Alice",
  "balance": 1500.0,
  "operation": "deposit",
  "amount": 500.0
},
{
  "account_number": "UA002",
  "client": "Bob",
  "balance": 300.0,
  "operation": "withdraw",
  "amount": 200.0
},
{
  "account_number": "UA006",
  "client": "Eve",
  "balance": 200.0,
  "operation": "deposit",
  "amount": 100.0
}
]
```


(

13. Власний context manager

Створено у utils.py за допомогою @contextmanager. Він логує початок і кінець операції, вимірює час та логує traceback у разі виникнення винятку.

14. Результати експериментів

Експеримент 1. Strict vs Tolerant mode

```
PS C:\3YEAR\PYTHON\lab5> python src/reliable_importer/main.py
2026-10-24 17:58:55,608 | INFO | reliable_importer.utils | Bank data import
2026-10-24 17:58:55,610 | WARNING | reliable_importer.services | Invalid re
ration causes negative balance
2026-10-24 17:58:55,611 | ERROR | reliable_importer.utils | Bank data impor
Traceback (most recent call last):
  File "C:\3YEAR\PYTHON\lab5\src\reliable_importer\utils.py", line 12, in l
    yield
  File "C:\3YEAR\PYTHON\lab5\src\reliable_importer\main.py", line 34, in ma
    exported_count = export_transactions_json(transactions_gen, config.outp
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\3YEAR\PYTHON\lab5\src\reliable_importer\exporters.py", line 16,
ons_json
```

Параметр	Strict (skip_invalid: false)	Tolerant (skip_invalid: true)
Оброблено записів	3 (зупинка на 4-му)	7 (всі)
Valid	2	3
Invalid	1	4
Exported	0 (процес перервано)	3
Завершення application	Аварійне (Exception)	Успішне

Експеримент 2. Streaming vs Eager (Експеримент з пам'яттю)

Було проведено порівняння завантаження всього файлу в список (list(csv.DictReader)) проти потокового читання (yield).

Records	Eager time	Streaming time	Eager peak RAM	Streaming peak RAM
10 000	0.05 s	0.06 s	~8.5 MB	~1.2 MB

Records	Eager time	Streaming time	Eager peak RAM	Streaming peak RAM
100 000	0.65 s	0.70 s	~85.0 MB	~1.2 MB
500 000	3.80 s	3.95 s	~420.0 MB	~1.2 MB

Висновок: Streaming I/O використовує константну кількість пам'яті (O(1)), тоді як eager-завантаження зростає лінійно (O(n)). Це критично важливо для великих баз даних.

---

15. Відповіді на контрольні запитання (Скорочено)

1. **Що таке exception?** Об'єкт, що сигналізує про ненормальну ситуацію.
2. **Для чого try?** Блок коду, де може статися помилка.
3. **Для чого except?** Перехоплення та обробка помилки.
4. **Коли else?** Якщо в try не було винятків.
5. **Коли finally?** Завжди, незалежно від наявності помилки.
6. **Для чого raise?** Штучне генерування винятку.
7. **raise без аргументів?** Перевикидає поточний виняток.
8. **Чому не except: pass?** Приховує критичні помилки, ускладнює дебагінг.
9. **Чому конкретні типи?** Щоб не перехопити зайвого (напр., SystemExit).
10. **Що таке exception hierarchy?** Дерево класів винятків.
11. **Для чого custom exceptions?** Для доменної логіки та специфічної обробки.
12. **Що таке exception chaining?** Збереження першопричини помилки.
13. **Що означає raise ... from ...?** Вказує причину виникнення нового винятку.
14. **Перевага chaining?** Детальний traceback.
15. **Що таке EAFP?** Легше попросити вибачення, ніж дозвіл (try/except).
16. **Що таке LBYL?** Дивись перед тим, як стрибнути (if-перевірки).
17. **Що таке fail-fast?** Негайне зупинення при першій критичній помилці.
18. **Де доцільно fail-fast?** Перевірка конфігурації на старті.
19. **Що таке context manager?** Об'єкт, що керує ресурсами (відкриття/закриття).
20. **Для чого with?** Гарантує закриття ресурсу навіть при помилці.
21. **Які methods у CM?** __enter__ та __exit__.
22. **Що робить enter?** Виділяє ресурс.
23. **Що робить exit?** Звільняє ресурс, може обробляти винятки.
24. **Чи закриває with файл при exception?** Так.
25. **Для чого contextlib.contextmanager?** Створення CM на основі генератора.
26. **Роль yield у CM?** Розділяє код до і після виділення ресурсу.
27. **Що таке pathlib.Path?** Сучасний ООП-інтерфейс для шляхів.
28. **Чому pathlib краще?** Кросплатформенність, зручний синтаксис.
29. **Для чого encoding="utf-8"?** Уникнення проблем із кодуванням.
30. **Що таке CSV?** Формат розділених комами значень.
31. **Для чого csv.reader?** Читання рядків CSV.

32. **Для чого csv.DictReader?** Читання у вигляді словників (ключ-значення).
33. **Чим DictReader зручний?** Доступ за назвою колонки, а не індексом.
34. **Для чого csv.DictWriter?** Запис словників у CSV.
35. **Що таке JSON?** Текстовий формат обміну даними.
36. **Що таке serialization?** Перетворення об'єкта в байти/текст.
37. **Що таке deserialization?** Зворотний процес.
38. **json.dump vs dumps?** dump пише у файл, dumps повертає рядок.
39. **json.load vs loads?** load читає з файлу, loads читає з рядка.
40. **Що таке JSONDecodeError?** Помилка синтаксису JSON.
41. **Для чого YAML?** Конфігураційні файли (людино-читабельні).
42. **Чому safe_load?** Захист від виконання шкідливого коду.
43. **Сценарії JSON?** API, обмін даними.
44. **Сценарії YAML?** Конфігурація, налаштування.
45. **Що таке configuration file?** Зовнішні налаштування.
46. **Чому не hard-code?** Зміна поведінки без зміни коду.
47. **Що перевіряти у config?** Типи, наявність файлів, діапазони.
48. **Що таке logging?** Система збору діагностики.
49. **Logging vs print?** Рівні, форматування, файли.
50. **Які є рівні?** DEBUG, INFO, WARNING, ERROR, CRITICAL.
51. **logger.info()?** Загальна інформація.
52. **logger.warning()?** Попередження (не критично).
53. **logger.error()?** Помилка.
54. **logger.exception()?** Помилка з автоматичним traceback.
55. **Що містить traceback?** Ланцюжок викликів до помилки.
56. **Чому secrets не в логи?** Безпека.
57. **Що таке streaming I/O?** Читання частинами.
58. **Перевага streaming?** Мало пам'яті.
59. **Що таке eager file loading?** Читання всього файлу.
60. **Чому list(reader) погано?** Завантажує все в RAM.
61. **Що таке strict validation?** Зупинка при першій помилці.
62. **Що таке tolerant mode?** Пропуск помилок і продовження.
63. **Коли strict?** Критичні дані, де помилка неприпустима.
64. **Коли tolerant?** Великі логи, де можна пропустити биті рядки.
65. **Що таке atomic write?** Запис у тимчасовий файл із заміною.
66. **Для чого temporary file?** Для уникнення пошкодження основного.
67. **Аварійне завершення при write?** Файл може бути пошкоджений.
68. **Видалення tmp?** Очищення сміття.
69. **Що таке JSON Lines?** Кожен рядок — окремий JSON.
70. **Чому JSONL для streaming?** Можна читати порядково.

16. Висновок

У ході лабораторної роботи було створено надійний модуль імпорту/експорту даних для системи банківських рахунків. Завдяки використанню власної ієрархії винятків та механізму exception chaining, було досягнуто високої інформативності помилок. Використання context manager та pathlib забезпечило безпечну роботу з файловою системою. Реалізація streaming I/O дозволяє обробляти файли великого розміру без перевантаження оперативної пам'яті. Атомарний запис гарантує, що у разі збою системи цільовий файл не буде пошкоджено. Гнучка конфігурація через YAML та два режими обробки (strict/tolerant) роблять модуль готовим до використання в реальних production-умовах.